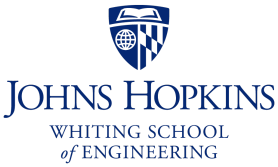
© Guven

Unsupervised Learning



Background

> "Statistics is the grammar of science." Karl Pearson

Unsupervised learning is a type of machine learning method that involves learning patterns from input data where the **ground truth** is either unknown or not referred to.

The most common unsupervised learning method is cluster analysis, which is used for **exploratory** data analysis to find grouping(s) or hidden patterns in data. This grouping is also called a **partition** in Computer Science. The clusters are identified using a measure of **similarity** which is defined upon metrics such as Euclidean or probabilistic distance. The cluster analysis groups data that has not been labeled, not classified, or not categorized. Instead of responding to feedback (target/dependent variable), cluster analysis identifies commonalities in the data. It reacts based on the presence or absence of such commonalities in each new piece of data.

The simplest form of clustering can be considered as the **binning** process as in data preprocessing, where the input numerical values are grouped according to the ranges of the bins.

- Supervised learning intends to infer a conditional probability distribution $P_X(x|y)$ conditioned on the labels y of input data
- Unsupervised learning intends to infer an a priori probability distribution $P_X(x)$

Unsupervised learning is the first form of learning, which is much more common in the brain.

Clustering is the primary method of unsupervised learning, which can answer questions like,

- What is the "normal" in the dataset? What is the baseline?
- Which data points are abnormal or anomalous in the dataset, if there are any?
- Is there a probability distribution that can model the input dataset?

- How many clusters are there naturally in the dataset?
- Anomaly detection

Types of clustering:

Hard clustering: Each data point can be assigned to one cluster only
Soft clustering: Also known as **fuzzy clustering**, where each data point can have a **membership** value to any cluster.

Major Clustering Algorithms

Distance/Similarity Measures

All clustering algorithms employ some form of similarity or a distance measure.

A distance metric, $d : X \times X \to [0, \infty)$, satisfies the following:

- $d(x, y) \geq 0$, non-negative
- $d(x, y) = 0 \iff x = y$, identity
- $d(x, y) = d(y, x)$, symmetry
- $d(x, y) \leq d(x, z) + d(z, y)$, triangle inequality

Define d_{ik} = the distance between x_i and the cluster center c_k where $k = 1, \ldots, K$, and $x \in \mathbb{R}^{N \mathbf{x} M}$ and $c_k \in \mathbb{R}^M$

Name	Distance
Euclidean distance	$d_{ik} = \sqrt{\sum_{j=0}^{M} (x_{ij} - c_{kj})^2}$
Manhattan distance	$d_{ik} = \sum_{j=0}^{M} \lvert x_{ij} - c_{kj} \rvert$
Chebyshev distance	$d_{ik} = \max_j \mid x_{ij} - c_{kj} \rvert$
Cosine similarity	$d_{ik} = \cos(\theta) = \frac{x_i \cdot c_k}{\lvert x_i \rvert \lvert c_k \rvert}$
Jaccard index	$d_{ik} = \frac{x_i \cap c_k}{x_i \cup c_k}$

K-means Clustering Algorithm

K-means clustering aims to partition N data points into K clusters in which each observation belongs to the cluster with the nearest mean (i.e., cluster centroid).

1. Randomly pick K data points from the input dataset as initial cluster centers, call μ_k where $k = 1, \ldots, K$.

2. Assign each data point x_i to the cluster via the nearest centroid μ_k where $k = 1, \ldots, K$.
3. Recompute (move) the centroids to the center of the data points of respective clusters.
4. Repeat steps 2 and 3 until the cluster assignments do not change or a user-defined tolerance or a pre-defined maximum number of iterations is reached.

Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN is a density-based clustering algorithm that groups points when they are closely packed together (points with many nearby neighbors). The algorithm marks the data points as outliers when they lie alone in low-density regions (whose nearest neighbors are too far away).

1. Find the points in the $\mathcal{E}$-neighborhood of every point, and identify the core points with more than `minimum Data Points` neighbors.
2. Find the connected components of core points on the neighbor graph, ignoring all non-core points.
3. Assign each non-core point to a nearby cluster if the cluster is an $\mathcal{E}$-neighbor; otherwise, assign it to noise.

Hierarchical Clustering

Hierarchical clustering seeks to build a hierarchy of clusters.

__Agglomerative__is a bottom-up approach where each observation starts in its own cluster, and pairs of clusters are merged as one moves up the hierarchy. Example draft algorithm:

1. Compute the distance matrix D of all data points, i.e. $N\mathbf{x}N$ size.
2. Represent each data point as a singleton cluster.
3. Merge the two closest clusters based on the distance between the most dissimilar (distant) members of those two clusters.
4. Update the similarity matrix.
5. Repeat steps 2-4 until one single cluster remains

__Divisive__is a top-down approach where all observations start in one cluster, and splits are performed recursively as one moves down the hierarchy. Example draft algorithm:

1. Assign all data points to one cluster.
2. For each cluster k where $k = 1, \ldots, K$, find the most dissimilar (distant) member x_{k1} to its centroid μ_k and find the most dissimilar member x_{k2} to

x_{k1}

3. Identify the k with the highest dissimilarity between x_{k1} and x_{k2} where $k = 1, \ldots, K$
4. Make x_{k1} and x_{k2} new cluster centers, remove the original cluster center μ_k, and re-assign all data points to the closest cluster centers.
5. Repeat steps 2-4 until each data point is a singleton.

Hierarchical methods cluster everything (either agglomerative or divisive) and let the user decide about the number of desired clusters since every possible number is already computed.

Expectation Maximization

The expectation maximization algorithm is an iterative method to find maximum likelihood or maximum a posteriori (MAP) estimates of parameters (such as mean and standard deviation) in statistical models. Example draft algorithm:

1. Start with an initial estimate of what each parameter might be.
2. Compute the likelihood that each parameter produces the data point, i.e., compute the probability of each data point belonging to a particular cluster, which is a Gaussian distribution.
3. Calculate weights for each data point, indicating whether it is more red or blue based on the likelihood of it being produced by a parameter. Combine the weights with the data (expectation).
4. Compute a better parameter estimate using the weight-adjusted data (maximization).
5. Repeat steps 2-4 until the parameter estimate converges (the process stops producing a different estimate).

Quality of Clustering - Metrics

Cluster Separation

Inter-cluster distance= $E = \frac{1}{\binom{K}{2}} \sum_{k=1}^{K} \sum_{l=k+1}^{K} d_{kl}$

where d_{kl} is the distance between cluster k centroid and cluster l centroid. Note that d_{kk} is zero. A good clustering algorithm maximizes this metric.

Cluster Cohesion

Intra-cluster distance= $E_k = \frac{1}{|x_i \in C_k|} \sum_{\forall x_i \in C_k}^{N} d_{ik}$

for cluster C_k, $k = 0, \ldots, K-1$

E_k is the average distance between the cluster centroid and the data points of that cluster k. A good clustering algorithm minimizes this metric.

Scoring Clustering

Regular accuracy, i.e. $\texttt{Accuracy} = \frac{\texttt{\#correct}}{\texttt{\#total}}$

Rand Index

After a clustering the cluster indexes may not match the original cluster indexes *literally*. The `contingency_matrix` function from the library `sklearn.metrics.cluster` can be used to map the resulting clusters to the original clusters (i.e. ground truth).

Given a set of n elements $S = \{o_1, \ldots, o_n\}$ and two partitions of S to compare, $X = \{X_1, \ldots, X_r\}$, a partition of S into r subsets, and $Y = \{Y_1, \ldots, Y_s\}$, a partition of S into s subsets, define the following:

- a, the number of pairs of elements in S that are in the same subset in X and in the same subset in Y
- b, the number of pairs of elements in S that are in different subsets in X and in different subsets in Y
- c, the number of pairs of elements in S that are in the same subset in X and in different subsets in Y
- d, the number of pairs of elements in S that are in different subsets in X and in the same subset in Y

The Rand index, $R = \frac{a+b}{a+b+c+d} = \frac{a+b}{\binom{n}{2}}$

Intuitively, $a + b$ can be considered as the number of **agreements** between X and Y and $c + d$ as the number of **disagreements** between X and Y.

Since the denominator is the total number of pairs, the Rand index represents the frequency of occurrence of agreements over the total pairs or the probability that X and Y will agree on a randomly chosen pair.

Worked Example

In [1]:
```
%env OMP_NUM_THREADS=1

%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72
import numpy as np
import pandas as pd
from sklearn.datasets import make_blobs

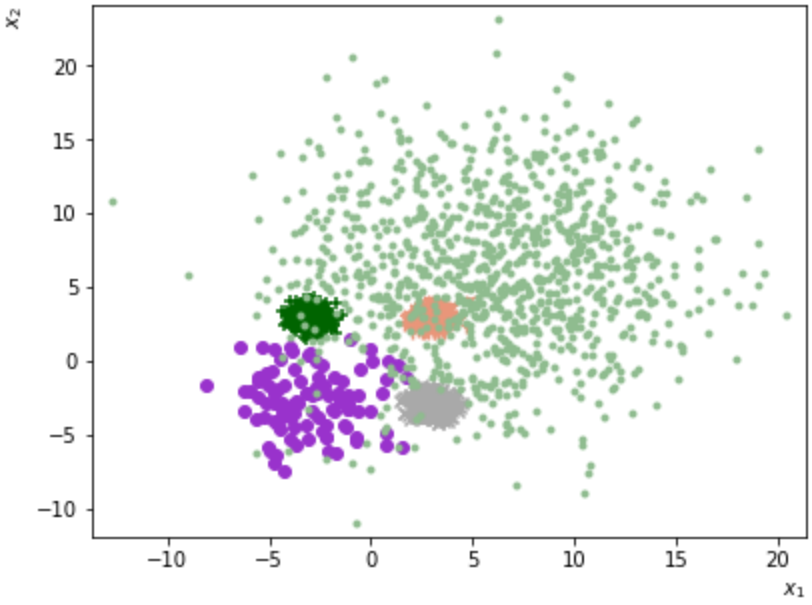

def get_minmax(_X, _m):  # _m = margin for visuals
    return _X[:,0].min()-_m, _X[:,0].max()+_m, _X[:,1].min()-_m, _X[:,1].max()+_m

def plotX(_X, _y, ax=None):  # Max 5 clusters/classes
    Colors = ['darkorchid', 'darkgreen', 'darkgrey', 'darksalmon', 'darkseagreen']
    Markers = ['o', '+', 'x', '1', '.']
    for c in range(max(_y)+1):
        plt.scatter(_X[_y==c,0], _X[_y==c,1], marker=Markers[c], color=Colors[c])
    #
    if ax is not None:
        ax.get_xaxis().set_ticks([])
        ax.get_yaxis().set_ticks([])
    #
    x1_min, x1_max, x2_min, x2_max = get_minmax(_X, 1)
    plt.xlim(x1_min, x1_max)
    plt.ylim(x2_min, x2_max)
    plt.xlabel(r'$x_1$', horizontalalignment='right', x=1.0)
    plt.ylabel(r'$x_2$', horizontalalignment='right', y=1.0)

# 5 classes with 2 features
X, y = make_blobs(n_samples=(100, 500, 500, 500, 1000), n_features=2, random_stat
                  cluster_std=[2, .5, .5, .5, 5],
                  centers=[(-3, -3), (-3, 3), (3, -3), (3, 3), (6, 6)])

plotX(X, y)
```

```
env: OMP_NUM_THREADS=1
```



In [2]:
```
from sklearn.cluster import KMeans
from sklearn.metrics.cluster import contingency_matrix
from sklearn.metrics import adjusted_rand_score

def kmeans_k(_n_clusters, _X, _y):
    km = KMeans(n_clusters=_n_clusters, init='random',
                n_init=10, max_iter=300, random_state=0)
    return km.fit_predict(_X), km.inertia_

def get_matchratio(_y, _y_km):
    # Compute contingency matrix
    cm = np.argmax(contingency_matrix(_y_km, _y), axis=1)
    # Map the clusters to the original using contingency above
    predicted = np.array([cm[c] for c in _y_km])
    # Compute match ratio, or accuracy
    match = sum([1 for y1, y2 in zip(_y, predicted) if y1 == y2])
    # Return the accuracy and mapped predictions
    return match/len(_y), predicted

EXPERIMENTS_K = (2,3,4,5,6,8,10)

for k in EXPERIMENTS_K:
    y_pred, inertia = kmeans_k(k, X, y)
    print(f'K-means with k={k:02d} performance, match rate={get_matchratio(y, y_p
          f'Adjusted Rand index={adjusted_rand_score(y, y_pred):.3f}')
```

```
K-means with k=02 performance, match rate=0.443, Adjusted Rand index=0.110
K-means with k=03 performance, match rate=0.603, Adjusted Rand index=0.343
K-means with k=04 performance, match rate=0.751, Adjusted Rand index=0.483
K-means with k=05 performance, match rate=0.795, Adjusted Rand index=0.472
K-means with k=06 performance, match rate=0.815, Adjusted Rand index=0.467
K-means with k=08 performance, match rate=0.860, Adjusted Rand index=0.490
K-means with k=10 performance, match rate=0.886, Adjusted Rand index=0.493
```

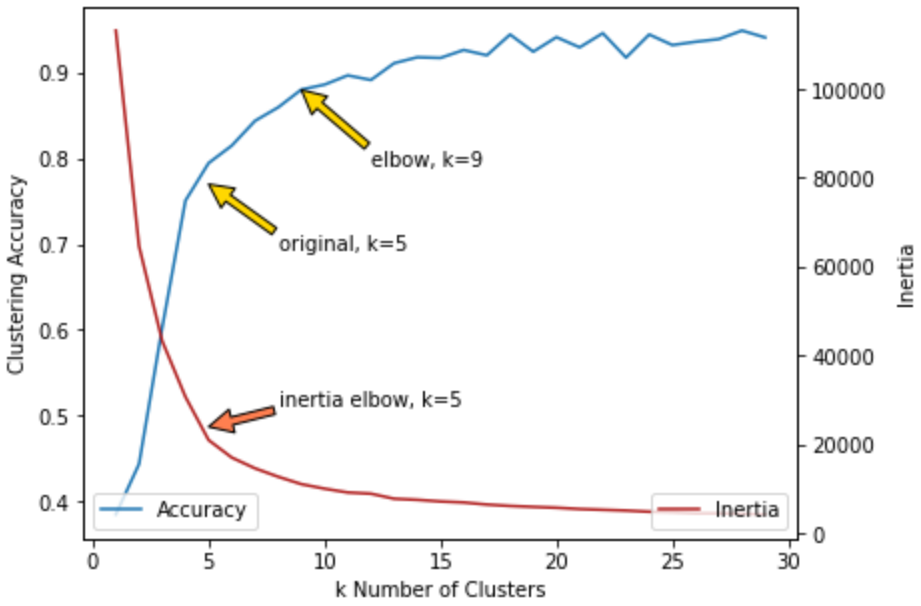
In [3]:
```
K = 30
# Compute accuracies for demonstration of the elbow
```

```
acc, inertia = [], []
for k in range(1,K):
    y_pred, iner = kmeans_k(k, X, y)
    ac, pred = get_matchratio(y, y_pred)
    acc += [ac]
    inertia += [iner]
```

In [4]:
```
# Accuracy
plt.plot(np.arange(1,K), acc, label='Accuracy')
plt.xlabel('k Number of Clusters')
k, ac = 9, 0.88
plt.annotate(f"elbow, k={k}", xy=(k, ac), xytext=(k+3, .9*ac), arrowprops=dict(fa
k, ac = 5, 0.77
plt.annotate(f"original, k={k}", xy=(k, ac), xytext=(k+3, .9*ac), arrowprops=dict
plt.ylabel('Clustering Accuracy')
plt.legend(loc='lower left')

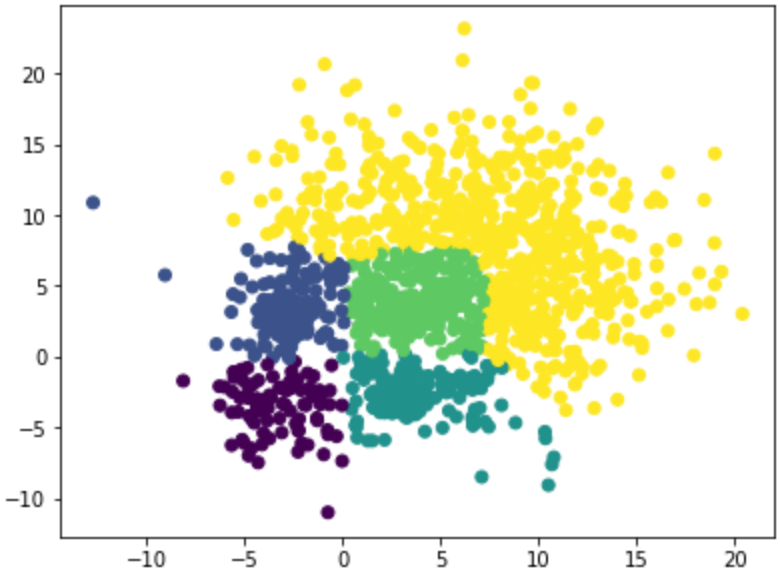
# Inertia
ax2 = plt.twinx()
ax2.plot(np.arange(1,K), inertia, color='firebrick', label='Inertia')
k, iner = 5, 24000
plt.annotate(f"inertia elbow, k={k}", xy=(k, iner), xytext=(k+3, 1.2*iner), arrow
plt.ylabel('Inertia')
plt.legend(loc='lower right')

plt.show()
```



In [5]:
```
# Try 7 clusters
k = 7
y_pred = get_matchratio(y, kmeans_k(k, X, y)[0])[1]

# Plot resulting clustering
plt.scatter(X[:,0], X[:,1], c=y_pred)
plt.show()
```



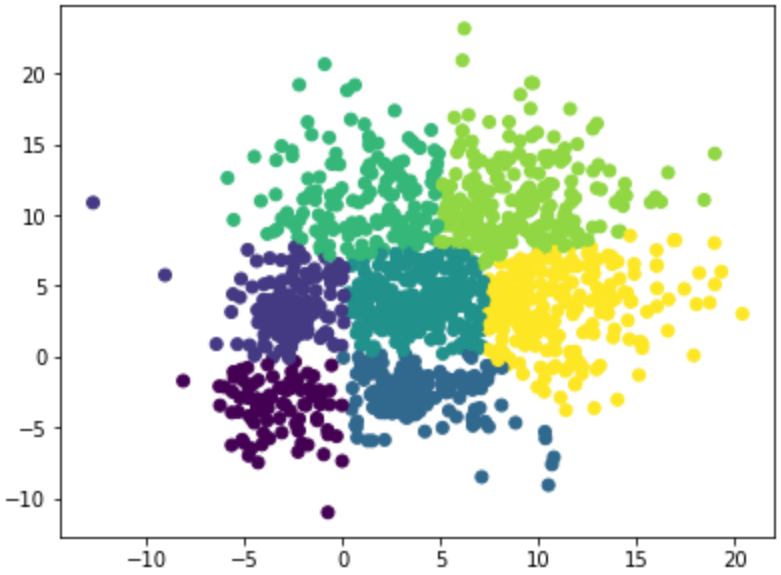
Important: Notice that by using contingency mapping, we are not reporting the true clustering error when we assign different cluster IDs to different parts of the same cluster. Thus, the following modification to the accuracy computation is realized, albeit probably penalizing the additional clusterings harshly.

Once a `kmeans` cluster is assigned to a particular original cluster, that original cluster ID is not used anymore. All remaining unassigned clusters are assigned to *new* cluster IDs so that error will accumulate.

In [6]:
```
def get_matchratio2(_y, _y_km):
    # Compute contingency matrix
    cm = np.argmax(contingency_matrix(_y_km, _y), axis=1)
    # Modification to contingency cluster mapping
    k_new = len(np.unique(_y))  # assumes y has consequent labels of 0, 1, 2, ...
    gt_dict = {k:0 for k in range(k_new)}
    for i, c in enumerate(cm):
        if gt_dict[c] == 0:
            gt_dict[c] = 1
        else:
            cm[i] = k_new
            k_new += 1
    # Map the clusters to the original using contingency above
    predicted2 = np.array([cm[c] for c in _y_km])
    # Compute match ratio, or accuracy
    match = sum([1 for y1, y2 in zip(_y, predicted2) if y1 == y2])
    # Return the accuracy and mapped predictions
    return match/len(_y), predicted2
```

In [7]:
```
# Cluster with k above
y_pred2 = get_matchratio2(y, kmeans_k(k, X, y)[0])[1]

# Plot resulting clustering
plt.scatter(X[:,0], X[:,1], c=y_pred2)
plt.show()
```



In [8]:
```
for k in EXPERIMENTS_K:
    y_pred, inertia = kmeans_k(k, X, y)
    print(f'K-means with k={k:02d} performance, match rate={get_matchratio2(y, y_
          f'Adjusted Rand index={adjusted_rand_score(y, y_pred):.3f}')
```

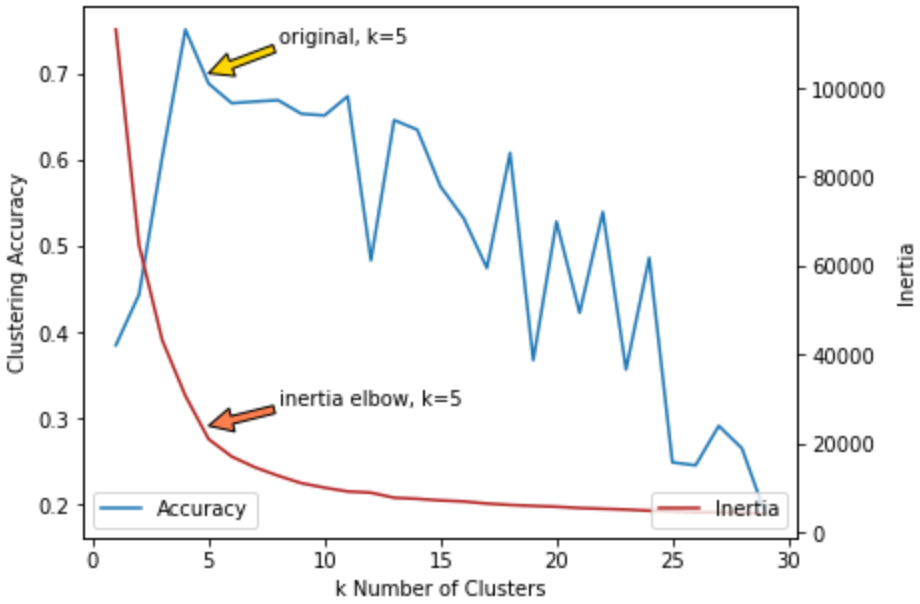
```
K-means with k=02 performance, match rate=0.443, Adjusted Rand index=0.110
K-means with k=03 performance, match rate=0.603, Adjusted Rand index=0.343
K-means with k=04 performance, match rate=0.751, Adjusted Rand index=0.483
K-means with k=05 performance, match rate=0.688, Adjusted Rand index=0.472
K-means with k=06 performance, match rate=0.665, Adjusted Rand index=0.467
K-means with k=08 performance, match rate=0.669, Adjusted Rand index=0.490
K-means with k=10 performance, match rate=0.651, Adjusted Rand index=0.493
```

In [9]:
```
K = 30
# Compute accuracies for demonstration of the elbow
acc, inertia = [], []
for k in range(1,K):
    y_pred, iner = kmeans_k(k, X, y)
    ac, pred = get_matchratio2(y, y_pred)
    acc += [ac]
    inertia += [iner]
```

In [10]:
```
# Accuracy
plt.plot(np.arange(1,K), acc, label='Accuracy')
plt.xlabel('k Number of Clusters')
k, ac = 5, 0.70
plt.annotate(f"original, k={k}", xy=(k, ac), xytext=(k+3, 1.05*ac), arrowprops=di
plt.ylabel('Clustering Accuracy')
plt.legend(loc='lower left')

# Inertia
ax2 = plt.twinx()
ax2.plot(np.arange(1,K), inertia, color='firebrick', label='Inertia')
k, iner = 5, 24000
plt.annotate(f"inertia elbow, k={k}", xy=(k, iner), xytext=(k+3, 1.2*iner), arrow
plt.ylabel('Inertia')
plt.legend(loc='lower right')
```

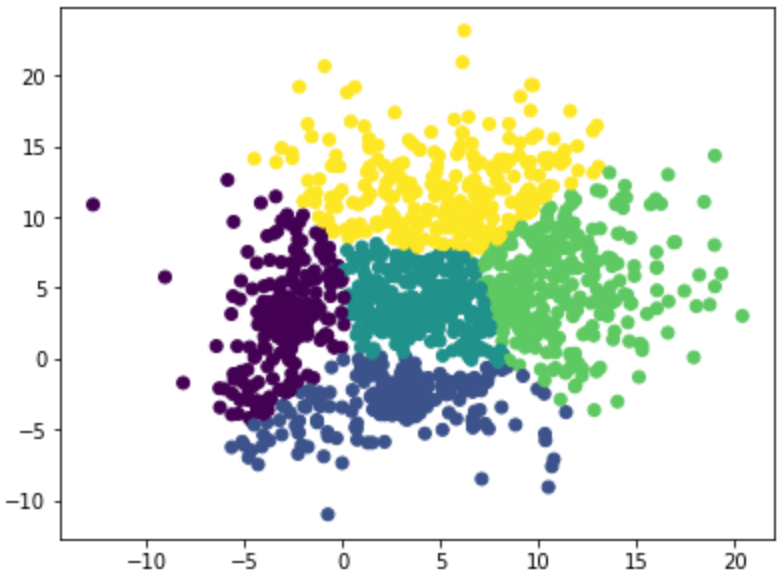
```
plt.show()
```



Cluster once more with k=5 and plot.

In [11]:
```
k = 5
# Cluster with k above
y_pred2 = get_matchratio2(y, kmeans_k(k, X, y)[0])[1]

# Plot resulting clustering
plt.scatter(X[:,0], X[:,1], c=y_pred2)
plt.show()
```



Now that we have an idea about how clustering results in this dataset, let's try other algorithms.

In [12]:
```
from sklearn.cluster import SpectralClustering

def spectral_k(_n_clusters, _X, _y):  # To keep the same API
    return (SpectralClustering(n_clusters=_n_clusters).fit_predict(_X), None)

for k in EXPERIMENTS_K:
    y_pred, inertia = spectral_k(k, X, y)
    print(f'Spectral clustering with k={k:02d} performance, match rate={get_match
          f'Adjusted Rand index={adjusted_rand_score(y, y_pred):.3f}')
```

```
Spectral clustering with k=02 performance, match rate=0.384, Adjusted Rand index=-0.
001
Spectral clustering with k=03 performance, match rate=0.543, Adjusted Rand index=0.2
39
Spectral clustering with k=04 performance, match rate=0.712, Adjusted Rand index=0.5
42
Spectral clustering with k=05 performance, match rate=0.497, Adjusted Rand index=0.3
25
Spectral clustering with k=06 performance, match rate=0.385, Adjusted Rand index=0.3
09
Spectral clustering with k=08 performance, match rate=0.222, Adjusted Rand index=0.0
27
Spectral clustering with k=10 performance, match rate=0.302, Adjusted Rand index=0.0
08
```

In [13]:
```
from sklearn.cluster import AgglomerativeClustering

def agglo_k(_n_clusters, _X, _y):  # To keep the same API
    return (AgglomerativeClustering(n_clusters=_n_clusters).fit_predict(_X), None

for k in EXPERIMENTS_K:
    y_pred, inertia = agglo_k(k, X, y)
    print(f'Agglomerative clustering with k={k:02d} performance, match rate={get_
          f'Adjusted Rand index={adjusted_rand_score(y, y_pred):.3f}')
```

```
Agglomerative clustering with k=02 performance, match rate=0.437, Adjusted Rand inde
x=0.097
Agglomerative clustering with k=03 performance, match rate=0.628, Adjusted Rand inde
x=0.368
Agglomerative clustering with k=04 performance, match rate=0.820, Adjusted Rand inde
x=0.608
Agglomerative clustering with k=05 performance, match rate=0.736, Adjusted Rand inde
x=0.522
Agglomerative clustering with k=06 performance, match rate=0.736, Adjusted Rand inde
x=0.560
Agglomerative clustering with k=08 performance, match rate=0.662, Adjusted Rand inde
x=0.552
Agglomerative clustering with k=10 performance, match rate=0.711, Adjusted Rand inde
x=0.538
```

Example, Iris Dataset and Quality Metrics

Let's look at the Iris dataset once more and see what natural clusters would look like in 3D.

Note: Please copy and run the code below in a Jupyter notebook and rotate the 3D figure below using the `%matplotlib notebook` command.

In [14]:
```
import seaborn as sns
from sklearn.datasets import load_iris

# Locate and load the data file
iris = load_iris()

# Use a DataFrame for convenience
df_iris = pd.DataFrame(data= iris.data, columns= iris.feature_names, dtype=float)
df_iris['species'] = pd.Series([iris.target_names[v] for v in iris.target])

# Sanity check
df_iris.head()
```

Out[14]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In [15]:
```
plt.figure(figsize=(18, 3), dpi=72)

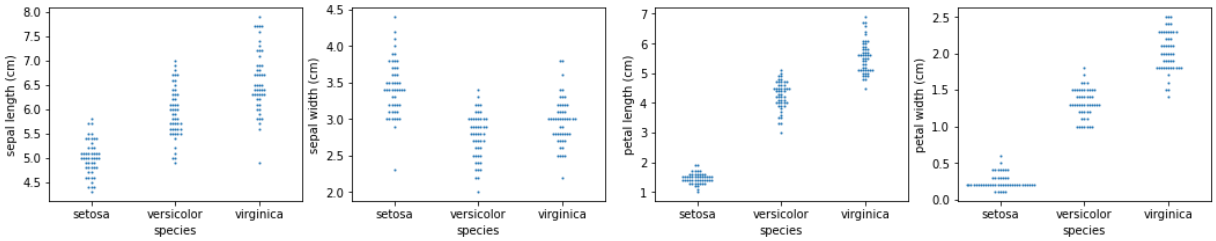
# Plot each feature versus categories
plt.subplot(1, 4, 1)
sns.swarmplot(x=df_iris["species"], y=df_iris["sepal length (cm)"], size=2)

plt.subplot(1, 4, 2)
sns.swarmplot(x=df_iris["species"], y=df_iris["sepal width (cm)"], size=2)

plt.subplot(1, 4, 3)
sns.swarmplot(x=df_iris["species"], y=df_iris["petal length (cm)"], size=2)

plt.subplot(1, 4, 4)
sns.swarmplot(x=df_iris["species"], y=df_iris["petal width (cm)"], size=2)

plt.show()
```

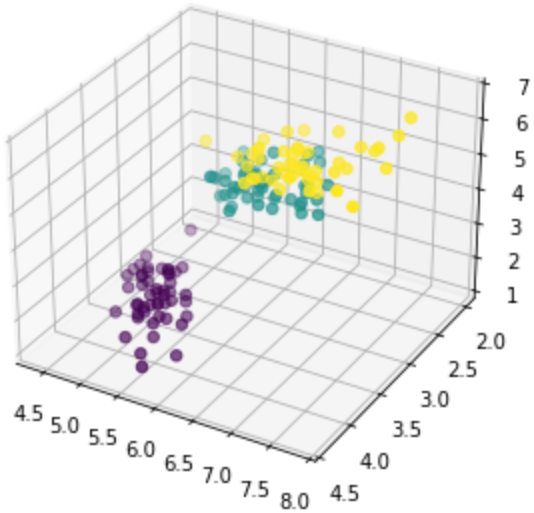


In [16]:
```
# %matplotlib notebook

from mpl_toolkits.mplot3d import Axes3D

def draw3d(df_ser1, df_ser2, df_ser3, df_ser_c):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.set_xlim3d(df_ser1.min()-0.1, df_ser1.max()+0.1)
    ax.set_ylim3d(df_ser2.min()-0.1, df_ser2.max()+0.1)
    ax.set_zlim3d(df_ser3.min()-0.1, df_ser3.max()+0.1)
    ax.set_ylim(ax.get_ylim()[::-1])
    ax.scatter(df_ser1, df_ser2, df_ser3, c=df_ser_c, s=30)

# Feature names as in the dataframe
draw3d(df_iris["sepal length (cm)"],
       df_iris["sepal width (cm)"],
       df_iris["petal length (cm)"],
       pd.factorize(df_iris['species'])[0])
```



Quality Metrics

Let's see how the effect of the number of clusters to K-means clustering and the error metrics.

In [17]:
```
from sklearn.cluster import KMeans

df_iris['species'] = df_iris['species'].replace({'setosa':0, 'versicolor':1, 'vir
```

```
X = df_iris.loc[:, df_iris.columns != 'species'].values
y = df_iris.loc[:, df_iris.columns == 'species'].values.ravel()

y_km, inertia = kmeans_k(3, X, y)  # tuple [1] returns the inertia

# Let's see how many clusters generated
np.unique(y_km)
```

Out[17]: array([0, 1, 2])

In [18]:
```
from collections import defaultdict
from functools import partial

def get_centroids(_X, _y):
    U = defaultdict(partial(np.ndarray, 0))
    Cn = defaultdict(int)
    for x, c in zip(_X, _y):
        U[c] = x if Cn[c] == 0 else U[c] + x
        Cn[c] += 1
    for c in U.keys():
        U[c] /= Cn[c]
    return U

def distance_inter_cluster(_U, _dist):
    # Compute inter-cluster errors
    K = np.max([int(c) for c in _U.keys()]) + 1
    if K < 2:
        return 0
    err, n = 0, 0
    for i in range(K):
        for j in range(i+1,K):
            err += _dist(_U[i], _U[j])
            n += 1
    return err / n

def distance_intra_cluster(_X, _y_km, _dist):
    # Compute intra-cluster errors
    U = get_centroids(_X, _y_km)
    En = defaultdict(partial(np.ndarray, 0))
    Cn = defaultdict(int)
    for x, c in zip(_X, _y_km):
        En[c] = _dist(x, U[c]) if Cn[c] == 0 else En[c] + _dist(x, U[c])
        Cn[c] += 1
    for c in U.keys():
        En[c] /= Cn[c]
    return En
```

In [19]:
```
# Define a distance measure
dist = lambda a, b : np.sqrt(np.sum((a-b)**2))
```

Question: What is the name of the distance measure above?

In [20]:
```
for k in (2,3,4,5):
```

```
    print(f'K-means with k={k} performance, match rate={get_matchratio2(y, kmeans
```

```
K-means with k=2 performance, match rate=0.667
K-means with k=3 performance, match rate=0.893
K-means with k=4 performance, match rate=0.727
K-means with k=5 performance, match rate=0.573
```

Let's look at more metrics:

In [21]:
```
for k in (2,3,4,5):
    y_km, intertia = kmeans_k(k, X, y)
    U = get_centroids(X, y_km)
    for c in sorted(U.keys()):
        print(f"{k}-means centroid {c}  [{' '.join([f'{x_:.2f}' for x_ in U[c]])}

    intercluster_vector = distance_inter_cluster(U, dist)
    intercluster_magnitude = np.linalg.norm(intercluster_vector)
    print(f'average inter-cluster distance= {intercluster_magnitude:.3f}')
    intracluster_vector = distance_intra_cluster(X, y_km, dist)
    for c in sorted(intracluster_vector.keys()):
        intracluster_magnitude = np.linalg.norm(intracluster_vector[c])
        print(f'cluster {c} intra-distance= {intracluster_magnitude:.3f}')

    print('-------------------------------------------------------------')
```

```
2-means centroid 0  [5.01 3.37 1.56 0.29]
2-means centroid 1  [6.30 2.89 4.96 1.70]
average inter-cluster distance= 3.929
cluster 0 intra-distance= 0.585
cluster 1 intra-distance= 1.003
-------------------------------------------------------------
3-means centroid 0  [5.01 3.43 1.46 0.25]
3-means centroid 1  [6.85 3.07 5.74 2.07]
3-means centroid 2  [5.90 2.75 4.39 1.43]
average inter-cluster distance= 3.391
cluster 0 intra-distance= 0.482
cluster 1 intra-distance= 0.720
cluster 2 intra-distance= 0.738
-------------------------------------------------------------
4-means centroid 0  [5.53 2.64 3.96 1.23]
4-means centroid 1  [5.01 3.43 1.46 0.25]
4-means centroid 2  [6.25 2.85 4.81 1.62]
4-means centroid 3  [6.91 3.10 5.85 2.13]
average inter-cluster distance= 2.829
cluster 0 intra-distance= 0.532
cluster 1 intra-distance= 0.482
cluster 2 intra-distance= 0.556
cluster 3 intra-distance= 0.700
-------------------------------------------------------------
5-means centroid 0  [7.48 3.12 6.30 2.05]
5-means centroid 1  [5.51 2.60 3.91 1.20]
5-means centroid 2  [6.21 2.85 4.75 1.56]
5-means centroid 3  [6.53 3.06 5.51 2.16]
5-means centroid 4  [5.01 3.43 1.46 0.25]
average inter-cluster distance= 2.802
cluster 0 intra-distance= 0.596
cluster 1 intra-distance= 0.522
cluster 2 intra-distance= 0.546
cluster 3 intra-distance= 0.453
cluster 4 intra-distance= 0.482
-------------------------------------------------------------
```

Question: By only looking at metrics above how do you like the number of clusters k to be?

Let's visualize clusters as in the following cells.

In [22]:
```
y_km, intertia = kmeans_k(3, X, y)
ac, y_pred = get_matchratio2(y, y_km)

# Helping sns.swarmplot for the clusters
df_ypred = pd.DataFrame()
df_ypred["cluster"] = pd.Series(y_pred)

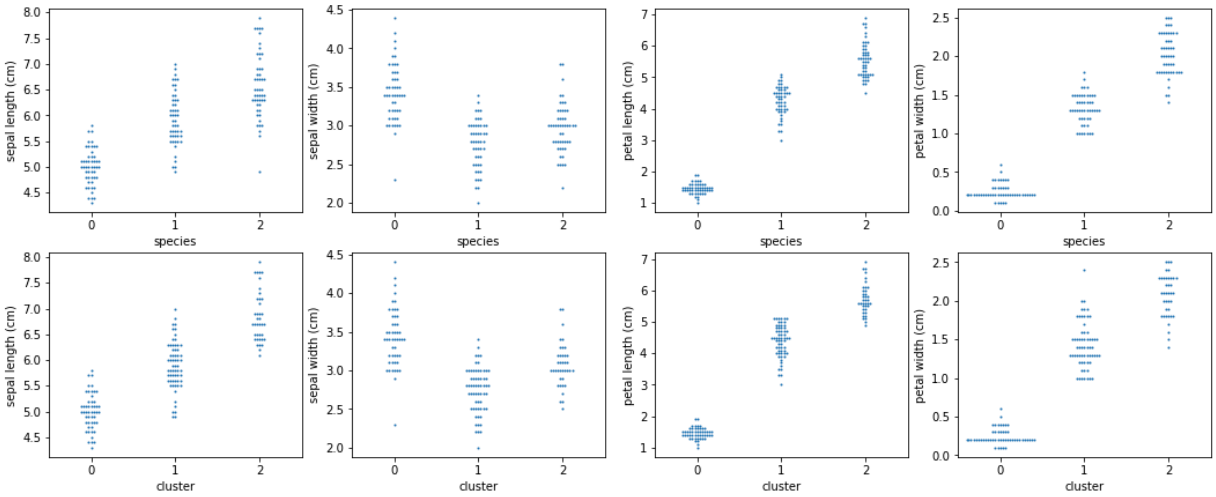
plt.figure(figsize=(18, 7), dpi=72)

# Plot each feature versus original categories
plt.subplot(2, 4, 1)
sns.swarmplot(x=df_iris["species"], y=df_iris["sepal length (cm)"], size=2)
plt.subplot(2, 4, 2)
```

```
sns.swarmplot(x=df_iris["species"], y=df_iris["sepal width (cm)"], size=2)
plt.subplot(2, 4, 3)
sns.swarmplot(x=df_iris["species"], y=df_iris["petal length (cm)"], size=2)
plt.subplot(2, 4, 4)
sns.swarmplot(x=df_iris["species"], y=df_iris["petal width (cm)"], size=2)

# Plot each feature versus cluster
plt.subplot(2, 4, 5)
sns.swarmplot(x=df_ypred["cluster"], y=df_iris["sepal length (cm)"], size=2)
plt.subplot(2, 4, 6)
sns.swarmplot(x=df_ypred["cluster"], y=df_iris["sepal width (cm)"], size=2)
plt.subplot(2, 4, 7)
sns.swarmplot(x=df_ypred["cluster"], y=df_iris["petal length (cm)"], size=2)
plt.subplot(2, 4, 8)
sns.swarmplot(x=df_ypred["cluster"], y=df_iris["petal width (cm)"], size=2)

plt.show()
```



Dendrograms for Hierarchical Clustering

The following cell demonstrates a simple dendrogram for uniform distances for three linkage types: {ward, complete, and single}. The clustering program could compute a full set of distances when clusters are joined, i.e., the distances between cluster centroids.

The x-axis is the individual data points, i.e., elements to be clustered. The y-axis levels are the clustering levels determined by the number of data points N.

In [23]:
```
from scipy.cluster.hierarchy import linkage, dendrogram

def plot_dendrogram_iris(_model, titletext='', **kwargs):
    # uniform distances
    distances = np.arange(_model.children_.shape[0])

    # number of observations contained in each clustering level
```

```
    no_of_observations = np.arange(2, _model.children_.shape[0]+2)

    # linkage matrix
    linkage_matrix = np.column_stack([_model.children_, distances, no_of_observat

    # dendrogram plotting
    dendrogram(linkage_matrix, **kwargs)
    plt.title(f'Iris dataset Dendrogram ({titletext})')
    plt.xlabel('sample index')

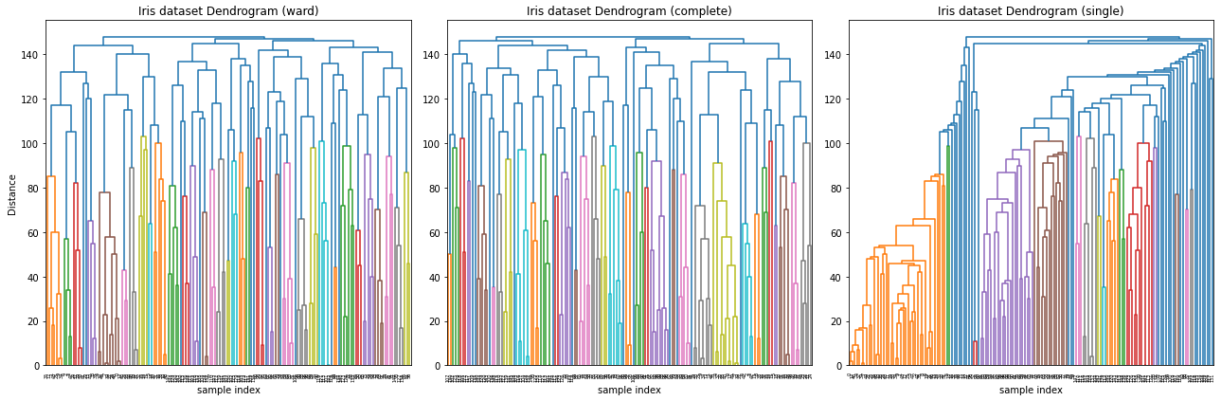
# Plot
plt.figure(figsize=(18, 6), dpi=72)

plt.subplot(1, 3, 1)
model = AgglomerativeClustering(linkage='ward')
model.fit_predict(X)
plot_dendrogram_iris(model, titletext='ward')
plt.ylabel('Distance')

plt.subplot(1, 3, 2)
model = AgglomerativeClustering(linkage='complete')
model.fit_predict(X)
plot_dendrogram_iris(model, titletext='complete')

plt.subplot(1, 3, 3)
model = AgglomerativeClustering(linkage='single')
model.fit_predict(X)
plot_dendrogram_iris(model, titletext='single')

plt.tight_layout()
plt.show()
```



In [24]:

```
from scipy.cluster.hierarchy import dendrogram

def plot_dendrogram_iris(_model, titletext='', **kwargs):
    # uniform distances
    distances = np.arange(_model.children_.shape[0])

    # number of observations contained in each clustering level
    no_of_observations = np.arange(2, _model.children_.shape[0]+2)

    # linkage matrix
```

```
    linkage_matrix = np.column_stack([_model.children_, distances, no_of_observat

    # dendrogram plotting
    dendrogram(linkage_matrix, **kwargs)
    plt.title(f'Iris dataset Dendrogram ({titletext})')
    plt.xlabel('sample index')

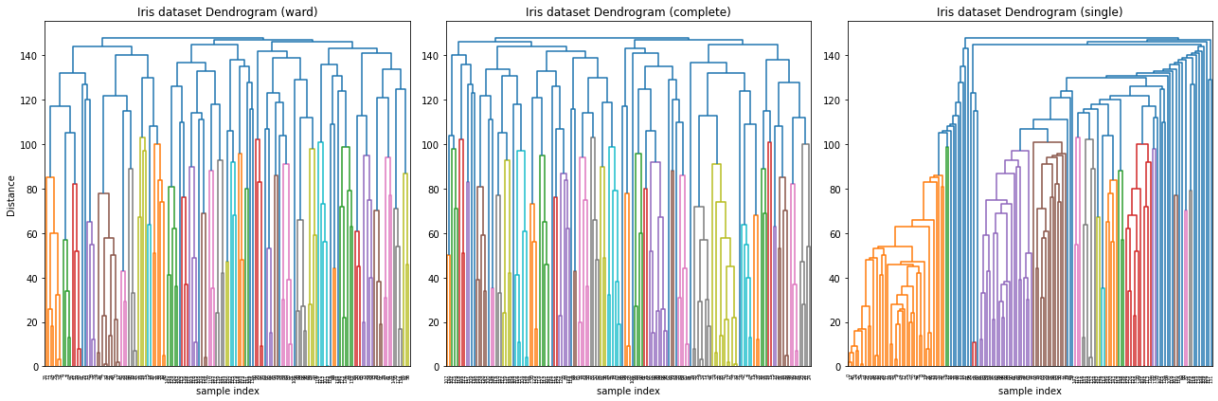
# Plot
plt.figure(figsize=(18, 6), dpi=72)

plt.subplot(1, 3, 1)
model = AgglomerativeClustering(linkage='ward')
model.fit_predict(X)
plot_dendrogram_iris(model, titletext='ward')
plt.ylabel('Distance')

plt.subplot(1, 3, 2)
model = AgglomerativeClustering(linkage='complete')
model.fit_predict(X)
plot_dendrogram_iris(model, titletext='complete')

plt.subplot(1, 3, 3)
model = AgglomerativeClustering(linkage='single')
model.fit_predict(X)
plot_dendrogram_iris(model, titletext='single')

plt.tight_layout()
plt.show()
```



References

1. Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.

Exercises

Exercise 1. In the dataset created in cell `1`:

- Change the location of the 5th class from `(6,6)` to `(1,1)` and report the performance. State your observations.
- Change the standard deviation of the 5th class from 5 to 0.5 and report the performance. State your observations.

Exercise 2. Build a dendrogram for the dataset created in cell `1`.

In [25]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
